

Obliczenia Naukowe

Rafał Wochna 279752

1 Zadanie 1

1.1 Cele Zadania

1. Wyznaczenie iteracyjnie epsilonów maszynowych (`macheps`) dla typów `Float16`, `Float32`, `Float64` i porównanie z wartościami funkcji `eps()` oraz danymi z pliku `float.h` języka C.
2. Wyznaczenie iteracyjnie liczb `eta` (najmniejszych dodatnich liczb maszynowych) dla typów `Float16`, `Float32`, `Float64` i porównanie z wartościami funkcji `nextfloat(0.0)`.
3. Wyznaczenie iteracyjnie maksymalnych wartości (`MAX`) dla typów `Float16`, `Float32`, `Float64` i porównanie z wartościami funkcji `floatmax()` oraz danymi z `float.h`.

1.2 Rozwiązanie

Kod implementujący obliczenia epsilon maszynowego, `eta`, maksymalnej wartości zmiennoprzecinkowej oraz minimalnej wartości znormalizowanej znajduje się w pliku `zad1.jl`. Funkcje `calculate_machine_epsilon`, `calculate_machine_eta` oraz `calculate_machine_max` obliczają odpowiednie wartości iteracyjnie dla typów `Float16`, `Float32` i `Float64`, porównując je z wbudowanymi funkcjami Julii (`eps`, `nextfloat`, `floatmax`, `floatmin`).

1.3 Wyniki obliczeń

Epsilon maszynowy (`macheps`)

Tabela 1: Porównanie wartości epsilonów maszynowych

Typ	Iteracyjnie	Wartość wbudowana	C <code>float.h</code>
<code>Float16</code>	0.000977	0.000977	–
<code>Float32</code>	$1.192\,092\,9 \times 10^{-7}$	$1.192\,092\,9 \times 10^{-7}$	$1.192\,092\,895\,507\,812 \times 10^{-7}$
<code>Float64</code>	$2.220\,446\,049\,250\,313 \times 10^{-16}$	$2.220\,446\,049\,250\,313 \times 10^{-16}$	$2.220\,446\,049\,250\,313 \times 10^{-16}$

Liczba eta

Tabela 2: Porównanie wartości eta

Typ	Iteracyjnie	nextfloat(0.0)
Float16	6.0×10^{-8}	6.0×10^{-8}
Float32	1.0×10^{-45}	1.0×10^{-45}
Float64	5.0×10^{-324}	5.0×10^{-324}

Wartości maksymalne

Tabela 3: Porównanie wartości maksymalnych

Typ	Iteracyjnie	floatmax()	C float.h
Float16	6.55×10^4	6.55×10^4	–
Float32	$3.402\,823\,5 \times 10^{38}$	$3.402\,823\,5 \times 10^{38}$	$3.402\,823\,466\,385\,289 \times 10^{38}$
Float64	$1.797\,693\,134\,862\,315\,7 \times 10^{308}$	$1.797\,693\,134\,862\,315\,7 \times 10^{308}$	$1.797\,693\,134\,862\,316 \times 10^{308}$

Minimalne wartości znormalizowane

Tabela 4: Porównanie minimalnych wartości znormalizowanych

Typ	floatmin()	Wartość z wykładu
Float32	$1.175\,494\,4 \times 10^{-38}$	1.2×10^{-38}
Float64	$2.225\,073\,858\,507\,201\,4 \times 10^{-308}$	2.2×10^{-308}

1.4 Interpretacja i wnioski

Zmienne wyznaczone w sposób iteracyjny dokładnie wyznaczają liczby macheps, eta oraz MAX. W Juli oraz C zmienne są takie same bo korzystają ze standardu IEEE 754.

- **Związek macheps z precyzją arytmetyki:** Liczba macheps jest równa precyzji arytmetyki ϵ ($\text{macheps} = \epsilon$). Epsilon maszynowy definiuje się jako precyzję arytmetyki, czyli maksymalny względny błąd zaokrąglenia.
- **Związek liczby eta z liczbą MINsub:** Liczba eta jest równa MINsub - czyli najmniejszej dodatniej liczbie zdenormalizowanej, co widać w Tabeli 2. Reprezentuje najmniejszą liczbę maszynową większą od zera, którą można reprezentować w systemie zmiennopozycyjnym.

- **Funkcje `floatmin()` i związek z `MINnor`:** Funkcje `floatmin(Float32)` i `floatmin(Float64)` zwracają `MINnor` - czyli najmniejszą dodatnią liczbę znormalizowaną. Jest ona równa w przybliżeniu wartości `floatmin()`.

2 Zadanie 2

2.1 Cel Zadania

Sprawdzenie eksperymentalne słuszności stwierdzenia Kahana, że epsilon maszynowy można otrzymać obliczając wyrażenie $3\left(\frac{4}{3} - 1\right) - 1$ w arytmetyce zmiennopozycyjnej. Weryfikacja metody dla wszystkich typów zmiennopozycyjnych: `Float16`, `Float32`, `Float64`.

2.2 Rozwiązanie

Kod implementujący obliczenie epsilon maszynowego metodą Kahana znajduje się w pliku `zad2.jl`. Funkcja `Kahan_machine_epsilon` oblicza epsilon dla typów `Float16`, `Float32` i `Float64`, wykorzystując precyzyjne operacje arytmetyczne. Wyniki są porównywane z wartościami zwracanymi przez wbudowaną funkcję `eps`.

2.3 Wyniki obliczeń

Tabela 5: Porównanie metody Kahana z funkcją `eps()`

Typ	Metoda Kahana	<code>eps()</code>
<code>Float16</code>	-0.000977	0.000977
<code>Float32</code>	$1.192\,092\,9 \times 10^{-7}$	$1.192\,092\,9 \times 10^{-7}$
<code>Float64</code>	$-2.220\,446\,049\,250\,313 \times 10^{-16}$	$2.220\,446\,049\,250\,313 \times 10^{-16}$

2.4 Interpretacja i wnioski

- **Reprezentacja binarna liczby $\frac{4}{3}$:** Liczba $\frac{4}{3}$ w systemie binarnym ma nieskończone rozwinięcie:

$$\frac{4}{3} = 1.0101\dots_2$$

co powoduje, że nie może być dokładnie reprezentowana w skończonej mantysie.

- **Wpływ zaokrąglania na znak wyniku:** W zależności od długości mantysy i algorytmu zaokrąglania, $\frac{4}{3}$ może być zaokrąglane w górę lub w dół, co tłumaczy różne znaki dla różnych typów.

- **Konieczność użycia wartości bezwzględnej w metodzie Kahana:** Aby metoda Kahana zwracała poprawny epsilon maszynowy, należy wziąć wartość bezwzględna z wyniku:

$$\text{macheps} = \left| 3 \left(\frac{4}{3} - 1 \right) - 1 \right|$$

3 Zadanie 3

3.1 Cele Zadania

1. Sprawdzenie eksperymentalne równomiernego rozmieszczenia liczb zmiennopozycyjnych w różnych przedziałach
2. Weryfikacja, że w przedziale $[1, 2]$ liczby są rozmieszczone równomiernie z krokiem $\delta = 2^{-52}$
3. Zbadanie jak zmienia się krok δ w różnych przedziałach: $[0.5, 1]$ oraz $[2, 4]$

3.2 Rozwiązanie

Kod realizujący wypisywanie reprezentacji bitowej liczb zmiennoprzecinkowych oraz kroków między kolejnymi liczbami w arytmetyce Float64 znajduje się w pliku `zad3.jl`. Funkcja `wypisz_bitstring` iteruje przez liczby w zadanym przedziale, wypisując ich reprezentację binarną (`bitstring`) oraz różnicę (`krok`) między kolejnymi liczbami oraz wypisuje `n` kroków. Wyniki są prezentowane dla przedziałów $[1, 2]$, $[0.5, 1]$ oraz $[2, 4]$.

3.3 Wyniki eksperymentalne

Tabela 6: Rozmieszczenie liczb w przedziale $[1, 2]$ - krok $\delta = 2^{-52}$

[illegible]

Tabela 7: Rozmieszczenie liczb w przedziale $[0.5, 1]$ - krok $\delta = 2^{-53}$

[illegible]

Tabela 8: Rozmieszczenie liczb w przedziale $[2, 4]$ - krok $\delta = 2^{-51}$

[illegible]

Wartości kroków:

- W przedziale $[1, 2]$: $\delta = 2^{-52} \approx 2.220446049250313 \times 10^{-16}$
- W przedziale $[0.5, 1]$: $\delta = 2^{-53} \approx 1.1102230246251565 \times 10^{-16}$ (2 razy mniejszy krok)
- W przedziale $[2, 4]$: $\delta = 2^{-51} \approx 4.440892098500626 \times 10^{-16}$ (2 razy większy krok)

3.4 Interpretacja i wnioski

- **Różnice bitowe:** Kolejne liczby w każdym przedziale różnią się wyłącznie inkrementacją mantysy o 1 bit, przy stałych bitach znaku i cechy, co potwierdza równomierność rozmieszczenia z krokiem δ . Zatem w danym przedziale nie istnieje żadna dodatkowa liczba zmiennopozycyjna pomiędzy nimi, co wynika ze skończonej precyzji mantysy.

4 Zadanie 4

4.1 Cele Zadania

1. Znaleźć eksperymentalnie w arytmetyce Float64 liczbę zmiennopozycyjną x w przedziale $1 < x < 2$, taką że $x \cdot (1/x) \neq 1$
2. Znaleźć najmniejszą taką liczbę w przedziale $(1, 2)$

4.2 Rozwiązanie

Funkcja `znajdz_liczbe` iteruje przez liczby w przedziale, dodając epsilon maszynowy (`eps(Float64)`) do wartości początkowej $x = 1.0$. Jeśli warunek $x \cdot (1/x) \neq 1$ zostanie spełniony, funkcja zwraca tę najmniejszą liczbę. Jeśli żadna liczba nie spełnia warunku, funkcja zwraca `nothing`. Kod znajduje się w pliku `zad4.jl`

4.3 Wyniki

Tabela 9: Wynik znalezienia liczby z niepoprawną operacją odwrotności

Parametr	Wartość
Znaleziona liczba x	1.000000057228997
$x \cdot (1/x)$	0.9999999999999999

4.4 Interpretacja i wnioski

- **Znalezienie liczby:** Najmniejszą liczbą w przedziale $(1, 2)$ dla której $x \cdot (1/x) \neq 1$ jest $x = 1.000000057228997$.
- **Przyczyna błędu:** Błąd wynika z kumulacji błędów zaokrągleń w dwóch operacjach:
 - Operacja odwrotności $1/x$ wprowadza błąd zaokrąglenia
 - Mnożenie $x \cdot (1/x)$ wzmacnia ten błąd
 - W wyniku otrzymujemy wartość 0.9999999999999999 zamiast dokładnie 1
- **Wartość błędu:** Różnica wynosi $1.110\,223\,024\,625\,156\,5 \times 10^{-16}$, co odpowiada $\epsilon/2$ (połowa macheps).
- **Znaczenie praktyczne:** Ten przykład demonstruje, że nawet proste operacje algebraiczne mogą dawać nieoczekiwane wyniki w arytmetyce zmiennopozycyjnej.

5 Zadanie 5

5.1 Cele Zadania

1. Zaimplementować cztery różne algorytmy obliczania iloczynu skalarnego wektorów x i y :

$$x = [2.718281828, -3.141592654, 1.414213562, 0.5772156649, 0.3010299957]$$

$$y = [1486.2497, 878366.9879, -22.37492, 4773714.647, 0.000185049]$$

2. Porównać wyniki dla typów `Float32` i `Float64`
3. Przeanalizować wpływ kolejności sumowania na dokładność wyniku

5.2 Rozwiązanie

Kod obliczający iloczyn skalarny dwóch wektorów różnymi metodami znajduje się w pliku `zad5.jl`. Funkcje `ForwardSum`, `BackwardSum`, `HighestToLowestSum` oraz `LowestToHighestSum` implementują sumowanie w różnych kolejnościach: od początku do końca, od końca do początku, od największego do najmniejszego oraz od najmniejszego do największego. Wyniki są obliczane dla typów `Float32` i `Float64`, a następnie wypisywane w konsoli.

5.3 Wyniki

Tabela 10: Wyniki obliczeń iloczynu skalarnego dla różnych algorytmów

Algorytm	Float32	Float64
Forward Sum	-0.4999443	1.0251881368296672e-10
Backward Sum	-0.4543457	-1.5643308870494366e-10
Highest to Lowest Sum	-0.5	0.0
Lowest to Highest Sum	-0.5	0.0

Wartość referencyjna: $-1.00657107000000 \cdot 10^{-11}$

5.4 Interpretacja i wnioski

- **Wpływ precyzji:** Wyniki dla `Float64` są znacznie bliższe wartości referencyjnej niż dla `Float32`, co pokazuje znaczenie użycia wyższej precyzji w obliczeniach.
- **Praktyczne znaczenie:** Eksperyment demonstruje, że kolejność sumowania może znacząco wpłynąć na dokładność wyniku, szczególnie w obliczeniach z pojedynczą precyzją. Wyniki iloczynu są obciążone błędem.

6 Zadanie 6

6.1 Cele Zadania

1. Obliczyć wartości funkcji $f(x) = \sqrt{x^2 + 1} - 1$ i $g(x) = \frac{x^2}{\sqrt{x^2+1}+1}$ dla malejących wartości x
2. Porównać wyniki obu funkcji matematycznie równoważnych

6.2 Rozwiązanie

Kod obliczający dla arytmetyki Float64 wartości funkcji $f(x) = \sqrt{x^2 + 1} - 1$ oraz $g(x) = \frac{x^2}{\sqrt{x^2+1}+1}$ dla $x = 8^{-i}$ ($i = 1, 2, \dots, 10$) znajduje się w pliku `zad6.jl`.

6.3 Wyniki

Tabela 11: Porównanie wartości funkcji $f(x)$ i $g(x)$ dla malejących x

x	$f(x)$	$g(x)$
8^{-1}	0.0077822185373186414	0.0077822185373187065
8^{-2}	0.00012206286282867573	0.00012206286282875901
8^{-3}	1.9073468138230965e-6	1.907346813826566e-6
8^{-4}	2.9802321943606103e-8	2.9802321943606116e-8
8^{-5}	4.656612873077393e-10	4.6566128719931904e-10
8^{-6}	7.275957614183426e-12	7.275957614156956e-12
8^{-7}	1.1368683772161603e-13	1.1368683772160957e-13
8^{-8}	1.7763568394002505e-15	1.7763568394002489e-15
8^{-9}	0.0	2.7755575615628914e-17
8^{-10}	0.0	4.336808689942018e-19

6.4 Interpretacja i wnioski

- **Przyczyna różnic:** Różnice wynikają z odejmowania dwóch bliskich sobie liczb $\sqrt{x^2 + 1} \approx 1$, które prowadzi do utraty cyfr znaczących. Działanie to jest numerycznie niestabilne i powoduje duże błędy względne.
- **Praktyczne znaczenie:** Eksperyment demonstruje, że pomimo matematycznej równoważności $f(x) = g(x)$, dla $x < 8^{-8}$ funkcja $f(x)$ zwraca wyniki 0.0, które są całkowicie błędne. Funkcja $g(x)$ zachowuje poprawność dzięki uniknięciu odejmowania bliskich sobie wartości poprzez odpowiednie przekształcenie algebraiczne.

7 Zadanie 7

7.1 Cele Zadania

1. Obliczyć przybliżoną wartość pochodnej funkcji $f(x) = \sin x + \cos 3x$ w punkcie $x_0 = 1$ metodą różnic skończonych
2. Przeanalizować błędy przybliżenia dla malejących wartości $h = 2^{-n}$ gdzie $n = 0, 1, 2, \dots, 54$
3. Wyjaśnić zjawisko pogarszania się dokładności przy zbyt małych wartościach h
4. Przeanalizować zachowanie wyrażenia $1 + h$ w arytmetyce zmiennopozycyjnej
5. Zidentyfikować optymalną wartość h minimalizującą błąd przybliżenia

7.2 Rozwiązanie

Kod obliczający przybliżoną wartość pochodnej funkcji $f(x) = \sin(x) + \cos(3x)$ w punkcie $x_0 = 1$ oraz błędy $|f'(x_0) - \tilde{f}'(x_0)|$ dla $h = 2^{-n}$ ($n = 0, 1, \dots, 54$) znajduje się w pliku `zad7.jl`. Funkcja `f_exact_derivative` oblicza dokładną wartość pochodnej, a przybliżenie jest realizowane za pomocą różnicowego wzoru. Wyniki są wypisywane w konsoli w formie tabeli.

7.3 Metoda obliczeń

Przybliżoną wartość pochodnej obliczamy ze wzoru:

$$f'(x_0) \approx \tilde{f}'(x_0) = \frac{f(x_0 + h) - f(x_0)}{h}$$

gdzie:

$$f(x) = \sin x + \cos 3x$$

$$f'(x) = \cos x - 3 \sin 3x$$

$$x_0 = 1$$

$$\text{Dokładna wartość } f'(1) = \cos 1 - 3 \sin 3 \approx 0.11694228168853815$$

$$h = 2^{-n} \quad \text{dla } n = 0, 1, 2, \dots, 54$$

7.4 Wyniki obliczeń

Tabela 12: Obliczenia pochodnej dla różnych wartości h

h	$1 + h$	Przybliżona pochodna	Błąd
2^{-1}	2.0	2.0179892252685967	1.9010469435800585
2^{-2}	1.5	1.8704413979316472	1.753499116243109
2^{-3}	1.25	1.1077870952342974	0.9908448135457593
2^{-4}	1.125	0.6232412792975817	0.5062989976090435
2^{-5}	1.0625	0.3704000662035192	0.253457784514981
2^{-6}	1.03125	0.24344307439754687	0.1265007927090087
2^{-7}	1.015625	0.18009756330732785	0.0631552816187897
2^{-8}	1.0078125	0.1484913953710958	0.03154911368255764
2^{-9}	1.00390625	0.1327091142805159	0.015766832591977753
2^{-10}	1.001953125	0.1248236929407085	0.007881411252170345
2^{-11}	1.0009765625	0.12088247681106168	0.0039401951225235265
$\vdots$	$\vdots$	$\vdots$	$\vdots$
2^{-23}	1.000000119	0.11694276239722967	4.807086915192826e-7
2^{-24}	1.000000059	0.11694252118468285	2.394961446938737e-7
2^{-25}	1.000000029	0.116942398250103	1.1656156484463054e-7
2^{-26}	1.000000014	0.11694233864545822	5.6956920069239914e-8
2^{-27}	1.000000007	0.11694231629371643	3.460517827846843e-8
2^{-28}	1.000000003	0.11694228649139404	4.802855890773117e-9
2^{-29}	1.000000001	0.11694222688674927	5.480178888461751e-8
2^{-30}	1.000000000	0.11694216728210449	1.1440643366000813e-7
$\vdots$	$\vdots$	$\vdots$	$\vdots$
2^{-50}	1.000000000	0.0	0.11694228168853815
2^{-51}	1.000000000	0.0	0.11694228168853815
2^{-52}	1.000000000	-0.5	0.6169422816885382
2^{-53}	1.000000000	0.0	0.11694228168853815
2^{-54}	1.000000000	0.0	0.11694228168853815

7.5 Interpretacja i wnioski

- **Optymalna wartość h :** Najmniejszy błąd występuje dla $h = 2^{-28}$ ($\approx 3.7 \times 10^{-9}$), gdzie błąd wynosi 4.8×10^{-9} . Jest to punkt równowagi między błędem obcięcia a błędem zaokrąglania.
- **Pogarszanie się dokładności dla małych h :** Dla $h < 2^{-28}$ błąd zaczyna ponownie rosnąć pomimo dalszego zmniejszania h . Wynika to z:
 - **Błędu obcięcia:** Przy bardzo małych h , różnica $f(x_0 + h) - f(x_0)$ staje się bardzo mała, co prowadzi do utraty precyzji w obliczeniach.
 - **Błędu zaokrąglania:** Odejmowanie bliskich sobie wartości w arytmetyce zmiennopozycyjnej powoduje utratę dokładności.

- **Zachowanie $1+h$:** Dla $h < 2^{-23}$ ($\approx 1.2 \times 10^{-7}$) wyrażenie $1+h$ jest zaokrąglane do 1.0 w arytmetyce Float64. Wynika to z ograniczonej precyzji reprezentacji liczb zmiennoprzecinkowych w standardzie IEEE 754.
- **Błąd zaokrąglenia:** Dla bardzo małych h , różnica $f(x_0+h) - f(x_0)$ jest bliska zeru, co prowadzi do utraty precyzji w wyniku odejmowania. W arytmetyce zmiennopozycyjnej, operacje na liczbach bliskich sobie są szczególnie podatne na błędy zaokrąglenia.